

计算机网络实践指导书

TCP 在应用层的设计与实现

赵增华、石高涛

2026 年 8 月修订

1. 实践目的

TCP 是 Internet 协议体系中的经典传输层协议。通过阅读 TCP 标准、分析已有代码框架并在应用层复现 TCP 的基本功能，学生应达到以下目标：

1. 理解传输层协议的基本功能，掌握网络协议设计和实现的基本方法；
2. 理解面向连接概念，掌握连接建立、连接关闭及状态管理的设计与实现方法；
3. 理解可靠数据传输，掌握序号与确认、滑动窗口、丢包检测、快速重传、超时重传及缓冲区管理方法；
4. 理解流量控制，掌握通告窗口和零窗口探测的设计与实现方法；
5. 理解拥塞控制的基本原理，掌握基础 Reno 的慢启动、拥塞避免和丢包后窗口减小方法；
6. 具备阅读 Internet 协议标准、分析已有软件框架并将标准要求转化为代码的能力；
7. 具备使用抓包、日志、自动测试和性能实验验证网络协议的能力；
8. 能够使用 AI 编程和写作工具辅助代码分析、设计、开发、测试和技术报告撰写，并能够识别、修正和验证 AI 输出；
9. 提高独立工程实践、版本管理、技术表达和学术诚信意识。

2. 实践内容

本实践根据 TCP 相关标准，在应用层实现简化的 TCP 协议，称为 TJU_TCP。TJU_TCP 采用课程自定义报文头并承载在 UDP 之上，不与互联网标准 TCP 直接互操作。详细技术要求见《计算机网络实践说明书》。主要内容如下：

1. 使用 UDP 发送和接收 TJU_TCP 控制报文与数据报文；
2. 使用 tcconfig、tc/netem 或课程指定工具模拟丢包、时延和带宽限制；
3. 使用 Wireshark、tcpdump、日志及 trace 分析协议运行过程；
4. 使用 AI 编程工具辅助分析已有框架的代码架构；
5. 使用 AI 工具辅助完成任务分解、协议设计和设计文档撰写；
6. 使用 AI 编程工具辅助完成各阶段代码实现、测试设计和错误诊断；
7. 独立实现连接管理、可靠数据传输和流量控制；
8. 独立实现基础 Reno 拥塞控制，并通过 trace 解释 cwnd、sssthresh、rwnd 和丢包事件的关系；
9. 完成功能测试、性能测试、课程报告和个人答辩；
10. 在完成必做内容后，可以选择完整 Reno 快速恢复、NewReno、SACK/RACK 或 CUBIC 等挑战任务。

2.1 标准依据

- TCP 基本规范：RFC 9293；

- RTT 估计和重传超时：RFC 6298；
- 基础 Reno 与完整 Reno 拥塞控制：RFC 5681；
- SACK 与 SACK 丢包恢复：RFC 2018、RFC 6675；
- NewReno 快速恢复：RFC 6582；
- TCP 窗口扩展与时间戳：RFC 7323；
- RACK-TLP 丢包检测：RFC 8985；
- CUBIC 拥塞控制：RFC 9438；
- TCP 经典规范与历史对照：RFC 793；
- 其他要求以《计算机网络实践说明书》和课程平台发布内容为准。

2.2 实践组织方式

本实践采用个人项目制。每名学生使用独立代码仓库，独立完成框架分析、协议设计、编码实现、测试、报告和答辩。学生之间可以讨论通用网络原理、RFC 理解和工具使用方法，但不得交换源代码、完整设计文档、AI 生成结果、实验数据或报告内容。

3. 实践总体流程

本实践采用“AI 辅助—人工核验—编程实现—测试验证—过程记录”的闭环工作方式。

阶段	主要任务	主要成果
第一阶段：环境、框架分析与总体设计	环境搭建与基线运行；AI 辅助代码架构分析；任务分解和协议总体设计	环境记录、架构图、需求追踪表、设计文档；第 1 周周末提交
第二阶段：协议核心功能实现	连接管理；可靠数据传输与流量控制	阶段代码、自动测试、日志和抓包；第 2 周周末提交
第三阶段：基础拥塞控制、综合测试与成果提交	基础 Reno；联调、功能复测和性能测试；最终报告、代码复现和个人答辩	基础 Reno 及测试证据、最终代码、原始数据、图表、报告和展示材料；第 3 周周末提交
挑战任务：高级拥塞控制与丢包恢复	完整 Reno 快速恢复、NewReno、SACK/RACK 或 CUBIC，可选择一项或多项	设计、代码、独立测试、trace 和对比图表；按课程通知单独提交

三个阶段各包含若干子任务。每一阶段均应形成可运行、可回退、可解释的 Git 版本，并在当周周末统一提交该阶段全部成果。不得到课程结束时一次性提交全部代码和文档。基础 Reno 属于第三阶段必做内容；高级拥塞控制与丢包恢复挑战任务独立提交。

4. AI 辅助实践方法

4.1 AI 的定位

AI 是本课程的工程辅助工具，主要用于提高代码理解、设计检查、编程、测试和文档撰写效率。AI 不替代学生对 TCP 原理、代码架构和实验结果的理解，也不替代编译、运行、测试和答辩。

本课程评价的重点不是使用了多少 AI，而是学生能否：

1. 向 AI 提供清晰、必要且安全的上下文；
2. 判断 AI 输出是否符合 RFC、任务要求和现有框架；
3. 发现 AI 建议中的错误、遗漏和不适用条件；
4. 修改 AI 输出并通过工程证据验证；
5. 对最终提交的代码和文档作出完整解释。

4.2 各阶段的 AI 使用方式

阶段	推荐的 AI 辅助任务	必须进行的人工验证
第一阶段：框架分析	解释目录和文件、梳理调用关系、识别数据流、总结关键结构	定位源文件和函数；编译运行；通过日志或断点验证调用路径
第一阶段：需求与设计	提取功能要求、生成需求清单、检查状态机、数据结构、异常路径和并发风险	对照说明书和 RFC 逐项核对；说明设计依据；检查接口、字段和测试条件的一致性
第二阶段：编程实现	解释编译错误、生成局部代码建议、重构重复逻辑	逐行审查；编译；运行正常、异常和边界测试
第二阶段：测试调试	生成测试矩阵、分析日志和抓包、提出故障假设	独立复现问题；设计对照实验；排除错误诊断
第三阶段：基础 Reno	检查窗口更新公式、分析 cwnd 变化、生成 trace 分析建议	对照 RFC 5681 和说明书；构造慢启动、拥塞避免、RTT 及重复 ACK 丢包场景；核对原始 trace
第三阶段：性能分析	生成数据处理或绘图建议、检查实验变量	核实原始数据和脚本；重复实验；解释异常值
第三阶段：报告撰写	组织章节、改善表达、检查设计与实现的一致性	核实全部技术描述、图表、实验数据和引用
挑战任务（选做）	比较完整快速恢复、NewReno、SACK/RACK 和 CUBIC 的适用条件与实现方案	阅读对应 RFC；核对前置条件；与基础 Reno 开展可复现实验对比

4.3 推荐的 AI 交互原则

1. **限定范围**：明确要求 AI 只分析指定文件、函数或问题，避免一次处理整个仓库；
2. **要求证据**：要求 AI 给出结论所依据的文件、函数、变量、调用关系或标准条款；
3. **暴露不确定性**：要求 AI 区分确定结论、推断结论和无法确认的内容；
4. **先分析后编码**：先让 AI 说明问题、约束和方案，再请求局部代码建议；
5. **小步修改**：每次只完成一个可测试改动，避免生成大段难以审查的代码；
6. **先写测试条件**：在实现前明确输入、预期输出、异常条件和验收标准；
7. **比较方案**：要求 AI 说明不同方案的正确性、复杂度和风险，而不是只给出一个答案；
8. **保护数据**：仅提供完成当前任务必要且经过脱敏的内容。

4.4 AI 输出验证清单

对 AI 生成或修改的代码，至少检查：

- 是否遵守规定接口和可修改边界；
- 是否正确处理网络字节序、报文长度和校验和；
- 是否存在内存越界、空指针、资源泄漏或未初始化变量；
- 是否存在死锁、竞态、重复创建线程或定时器等问题；
- 是否正确处理丢包、重复、失序、超时和窗口为 0 等边界情况；
- 是否把接收窗口 `rwnd`、拥塞窗口 `cwnd` 和实际在途数据量混为一谈；
- `RTT`、`cwnd` 和 `ssthresh` 更新是否符合说明书规定，是否错误地从重传报文采样 `RTT`；
- 是否引入固定测试结果、跳过协议流程等“为通过测试而编码”的逻辑；
- 是否能够通过本地测试、线上测试和抓包结果共同验证；
- 学生是否能够独立解释并修改该段代码。

对 AI 生成或修改的文档，至少检查：

- 标准编号和技术表述是否正确；
- 设计、实现和实验结果是否相互对应；
- 图表是否来自本人实验；
- 是否存在虚构引用、虚构数据或未经验证的结论；
- 是否保留必要的 AI 使用说明。

4.5 AI 使用记录

学生应在框架分析、协议或模块设计、分阶段编程实现、测试/诊断、技术文档撰写与一致性检查五类环节中，各保留至少一项有实质意义的 AI 协作记录，但不要求提交完整对话。建议采用下表：

阶段	工具及版本	使用目的	AI 建议摘要	人工修改或拒绝内容	验证方法	关联提交/材料
示例： 第一阶段—框架分析	由学生填写	分析报文接收路径	由学生填写	由学生填写	源码定位+运行日志	Git 提交或报告章节

评分关注 AI 使用是否合理、验证是否充分、记录是否真实，不以工具品牌、对话次数或生成代码量评分。

5. 三周进度安排

5.1 总体安排

时间	主要内容	阶段成果与验收
第 1 周：第一阶段	子任务 1：环境搭建与基线运行； 子任务 2：AI 辅助代码架构分析； 子任务 3：任务分析与协议总体设计	环境记录、架构分析、需求追踪表、设计文档 V1、Git 版本； 第 1 周周末提交
第 2 周：第二阶段	子任务 1：连接管理； 子任务 2：可靠数据传输与流量控制	源代码、自动测试结果、日志或抓包、进度报告、Git 版本； 第 2 周周末提交
第 3 周：第三阶段	子任务 1：基础 Reno 拥塞控制； 子任务 2：联调、功能复测与性能测试； 子任务 3：最终报告、代码复现与个人答辩	基础 Reno 测试证据、最终代码、原始数据、绘图脚本、性能分析、课程报告、AI 使用说明和展示材料； 第 3 周周末提交

挑战任务在完成必做任务后开展，可选择完整 Reno 快速恢复、NewReno、SACK/RACK 或 CUBIC 中的一项或多项，不并入三个阶段的基本分值。

5.2 进度管理要求

1. 第一、第二、第三阶段分别在第 1 周、第 2 周、第 3 周周末提交；具体截止时间以课程平台通知为准；
2. 进度报告采用累积方式：在上一次报告基础上增加新阶段内容；
3. 每个正式阶段的提交应包含该阶段全部子任务，并对应明确的 Git 版本；
4. 未通过阶段测试时也应按时提交当前成果、失败证据和问题分析；
5. 延期提交按照课程评分规定处理；因不可抗力申请延期的，应按学校和课程要求履行手续；
6. 不允许在课程结束前集中补造 Git 记录、AI 记录或测试数据。

6. 三阶段实践指导

6.1 第一阶段：环境、框架分析与总体设计（第 1 周）

第一阶段由三个子任务组成，应在第 1 周周末统一提交全部成果。

6.1.1 子任务一：环境搭建与基线运行

工作内容

1. 按课程发布说明安装或导入指定运行环境；
2. 获取框架，核对版本号或校验值；
3. 在不修改源代码的情况下完成编译；
4. 分别启动服务端和客户端，验证 UDP 基本通信；
5. 验证网络参数配置工具、抓包工具和课程测试脚本；
6. 建立个人 Git 仓库并提交基线版本。

检查要点

- 操作系统、编译器和依赖版本是否符合要求；
- 两个运行实例的 IP 地址和端口是否正确；
- 是否存在防火墙、端口占用、权限或路径问题；
- 基线代码是否与教师发布版本一致；
- 能否在不依赖 IDE 状态的情况下通过命令复现编译和运行过程。

子任务成果

- 环境与工具版本表；
- 编译、运行和基础通信结果；
- 基线 Git 提交标识；
- 未解决问题。

6.1.2 子任务二：AI 辅助代码架构分析

建议按照以下顺序开展：

1. 查看目录、构建文件和接口文档；
2. 识别程序入口和服务端/客户端启动过程；
3. 从八个规定接口出发追踪调用关系；
4. 追踪一个控制报文和一个数据报文的完整路径；
5. 分析 `tju_tcp_t`、报文结构、缓冲区和状态变量；
6. 识别线程、锁、条件变量和定时器；
7. 标记已实现、待实现和限制修改部分；
8. 运行框架，通过日志、断点或临时诊断信息验证分析结论；
9. 修正 AI 工具给出的不准确或无法由代码支持的结论。

重点成果

1. 模块架构图；
2. 主要接口调用关系图；
3. 控制报文和数据报文的数据流；
4. 关键数据结构及其生命周期；
5. 架构风险与待确认问题；
6. AI 分析、人工核验和修正记录。

可以使用“文件/模块—主要职责—关键接口—是否允许修改”表辅助分析，但该表仅用于提高可追踪性，最终提交内容以说明书 v2 规定为准。

常见问题

- 只根据文件名推断功能，未阅读函数实现；
- 直接使用 AI 生成的架构图，但图中模块在代码中不存在；
- 混淆教师框架已经实现的功能和学生需要实现的功能；
- 只进行静态阅读，未通过运行验证调用关系；
- 架构图过于抽象，不能映射到具体文件、函数或数据结构。

6.1.3 子任务三：任务分析与协议总体设计

需求追踪

建立以下追踪表：

任务要求	标准依据	设计模块/ 状态	主要数据 结构	测试方法	实现状态
三次握手	RFC 9293 及说明书	由学生填写	由学生填写	丢包/时延下连接测试	未开始/进行中/完成

追踪表应覆盖说明书中的全部必做功能，并随着开发过程持续更新。

总体设计

总体设计至少说明：

1. 模块划分及其关系；
2. 报文处理入口和分发方法；
3. socket 状态及状态转换；
4. 发送端与接收端缓冲区；
5. 序号、确认号和窗口变量；
6. 重传定时器和 RTT/RT0 估计；
7. 流量控制中的 `rwnd` 与基础拥塞控制中的 `cwnd`、`ssthresh` 和 `FlightSize`；
8. 线程与同步机制；
9. 错误处理和资源释放；
10. 功能测试和性能测试思路

设计文档中的 AI 使用

AI 可以辅助生成设计初稿、检查遗漏和比较方案，但学生应：

- 逐项核对 RFC 和说明书；
- 删除超出课程范围或与框架不兼容的内容；
- 明确记录 AI 建议中被修改或拒绝的部分；
- 说明最终设计的取舍理由；
- 确保设计图、数据结构和后续代码一致。

6.1.4 第一阶段提交

第 1 周周末统一提交：环境与基线记录、架构分析材料、需求追踪表、协议设计文档 V1、对应 Git 版本、代表性 AI 协作与人工验证记录，以及未解决问题和第 2 周计划。

6.2 第二阶段：协议核心功能实现（第 2 周）

第二阶段由两个子任务组成，应在第 2 周周末统一提交全部成果。

6.2.1 子任务一：连接管理

连接建立

重点掌握：

1. 客户端和服务端的状态变化；
2. SYN、SYN-ACK 和 ACK 字段，以及 SYN 占用一个序列号的规则；
3. 32 位初始序列号 ISN 的选择：不同连接不得固定使用同一常量，推荐包含单调变化分量和本地不可预测分量；
4. 确认号表示对方下一个期望的序列号；
5. 握手报文丢失后的 RTO 重传，以及第三个 ACK 丢失后客户端对重复 SYN-ACK 再次返回有效 ACK；
6. 若使用小于 3 秒的初始 RTO 且 SYN 发生重传，连接建立后的数据阶段把 RTO 重新初始化为 3 秒；
7. tju_connect、tju_listen 和 tju_accept 的阻塞或唤醒关系；
8. 重复报文对状态机的影响。

建议先在无丢包环境下验证正常三次握手，再依次模拟 SYN、SYN-ACK 和最终 ACK 丢失。

说明书不要求实现同时建立连接、旧重复 SYN、连接建立期间崩溃恢复、半开放连接发现和 RST。设计文档应将这些内容明确列为教学范围裁剪，不应误写成已经实现。

连接关闭

重点掌握：

1. 双方先后关闭和同时关闭；
2. FIN 和 ACK 的序号空间，以及 ACK 与本端 FIN 可以合并的情况；
3. `tju_close` 之后不再接受新发送，CLOSE 之前提交的数据排列在 FIN 之前并得到可靠处理；
4. 关闭过程中的重复报文和超时重传；
5. FIN-WAIT、CLOSE-WAIT、CLOSING、LAST-ACK 和 TIME-WAIT 状态；
6. 主动关闭方等待 $2 \times \text{TJU_MSL}$ 后再释放连接；TJU_MSL 使用课程平台规定值；
7. 状态、线程、定时器和内存资源的正确释放。

实现时不要把“典型四个报文”写成固定报文数量。重点是两个方向分别由FIN关闭，并且每个FIN都得到确认。在测试一方不响应FIN时，至少应验证FIN超时重传和课程规定的最终终止条件。

AI 辅助建议

可让 AI 检查状态机是否遗漏转移、分析日志中的异常状态或生成边界测试清单。不得让 AI 通过删除状态检查、固定返回成功或识别测试环境等方式“修复”问题。

6.2.2 子任务二：可靠数据传输与流量控制

可靠数据传输的实现顺序建议

1. 明确字节序列号和确认号的语义；
2. 实现发送和接收缓冲区基本操作；
3. 实现无丢包条件下的数据分段、发送、接收和重组；
4. 实现累计确认和发送窗口滑动；
5. 按课程框架规定的存放位置、覆盖范围和字节序实现 checksum，并丢弃校验失败的报文；
6. 处理重复、重叠和失序报文，确保每个字节只向应用按序交付一次；
7. 实现 RFC 6298 的 RTT 估计、RTO 计算、Karn 算法和超时重传；
8. 实现三次重复 ACK 触发的基本快速重传；
9. 在逐渐增加的丢包率和时延下测试；
10. 验证不少于 100 MB 数据的可靠传输；
11. 检查 `tju_close` 返回前能否完成此前提交数据的可靠交付，或明确返回规定的失败状态。

RTT、RTO 和重传定时器

建议为每条连接维护 SRTT、RTTVAR、RTO 和一个针对最早未确认数据的重传定时器。基本计算如下：

1. 尚无 RTT 样本时，初始 $\text{RTO} = 1 \text{ s}$ ；

2. 第一个有效样本为 R 时: $SRTT=R$, $RTTVAR=R/2$, $RTO=SRTT+\max(G, 4\times RTTVAR)$;
3. 后续样本按照 RFC 6298 先更新 $RTTVAR$ 、再更新 $SRTT$, 最后重新计算 RTO ;
4. 除非使用时间戳等方式消除歧义, 不从已经重传的报文取得 RTT 样本;
5. 发送首个在途数据时启动定时器; 累计 ACK 推进时以当前 RTO 重新启动; 全部在途数据确认后停止;
6. 定时器超时重传最早未确认报文, 将 RTO 加倍后重新启动; 同一报文不得早于一个 RTO 被再次超时重传。

日志应能区分原始发送、快速重传和 RTO 重传, 并记录 RTT 样本是否被采用。AI 经常生成“每个报文一个永久线程/定时器”或“每次 ACK 都采样 RTT ”的实现, 必须结合 RFC 6298 和并发测试进行检查。

调试建议

- 为关键事件设计结构化日志: 发送、接收、ACK 推进、超时、重传和窗口变化;
- 日志应包含相对时间、连接标识、序号、确认号、长度、窗口和事件类型;
- 不要只看最终文件是否相同, 还应检查是否出现重复发送、无效重传、窗口停滞或线程失控;
- 先测试小文件和可控丢包场景, 再测试 100 MB 文件;
- 对 AI 提出的故障原因, 应设计对照实验验证, 不要根据单次日志直接接受结论。

常见问题

- 混淆报文序号和字节序号;
- ACK 推进条件错误, 过早释放发送缓冲区;
- 对失序报文处理不当, 导致累计确认错误;
- 为每个报文无控制地创建线程或定时器;
- 对重传报文重新采样 RTT , 导致 RTO 估计错误;
- 未在 ACK 推进时重启定时器, 或者在全部数据确认后仍保留活动定时器;
- RTO 超时后没有进行指数退避, 导致连续激进重传;
- 并发访问缓冲区或状态变量时缺少同步;
- 关闭连接时直接终止线程, 造成剩余数据丢失。

流量控制

重点实现:

1. 接收缓冲区可用空间计算: 发送缓冲区和接收缓冲区容量均不得小于 $5000\times SMSS$ 字节;
2. Advertised Window 字段更新: 未实现窗口扩展时通告 $\min(\text{可用空间}, 65535)$, 避免 16 位整数截断回绕;
3. 发送方有效窗口计算, 第二阶段暂由 $rwnd$ 约束; 第三阶段加入 $cwnd$ 后使用 $\min(rwnd, cwnd)$;
4. 接收方避免主动缩小窗口右边界, 发送方能够处理窗口缩小;
5. 零窗口处理和探测: 首个探测在零窗口持续一个 RTO 后发送, 后续间隔指数增长, 接收端以当前确认号和窗口响应;

6. 发送端和接收端的基本 SWS 避免机制，避免持续发送或通告过小窗口；
7. 窗口变化日志及图表。

建议设置较小接收缓冲区或降低接收处理速度，观察通告窗口逐渐减小、变为 0、探测并恢复的过程。实验中应区分流量控制窗口 `rwnd` 和第三阶段必做的拥塞窗口 `cwnd`。本课程不要求实现 Nagle 算法。

6.2.3 第二阶段提交材料

第 2 周周末统一提交：连接管理、可靠数据传输和流量控制源代码，对应 Git 版本，自动测试结果，必要的日志、抓包或 `trace`，累积进度报告，本阶段代表性 AI 协作与人工验证记录，以及未解决问题和第 3 周计划。

6.3 第三阶段：基础拥塞控制、综合测试与成果提交（第 3 周）

第三阶段由三个子任务组成，应在第 3 周周末统一提交全部成果。

6.3.1 子任务一：基础 Reno 拥塞控制

关键变量

- `SMSS`：发送端最大报文数据长度，不包含 `TJU_TCP` 头部；按说明书 v2，最大报文总长度为 1400 字节、固定头部为 20 字节时，`SMSS=1380` 字节；
- `rwnd`：接收端最近通告的接收窗口；
- `cwnd`：发送端拥塞窗口，以字节为单位；
- `ssthresh`：慢启动阈值，初始值设置为课程允许的最大接收窗口；
- `FlightSize`：已经发送但尚未累计确认的数据字节数。

发送端允许的在途数据量不得超过 $\min(rwnd, cwnd)$ 。不要用发送缓冲区容量代替 `FlightSize`，也不要把 `rwnd` 直接写入 `cwnd`。

推荐实现顺序

1. 在连接控制块中加入并初始化 `cwnd`、`ssthresh` 及拥塞控制状态；初始值使用课程平台发布的 RFC 5681 兼容配置；
2. 在发送判定中同时应用 `rwnd` 和 `cwnd`；
3. 实现慢启动：当 `cwnd < ssthresh` 时，每个累计确认新数据的 ACK 使 `cwnd` 增加，但单次增加不超过一个 `SMSS`；
4. 实现拥塞避免：当 `cwnd >= ssthresh` 时，使窗口约每个 RTT 增加一个 `SMSS`，且不得比 RFC 5681 的 Reno 规则更激进；
5. 实现 RTT 丢包响应：设置 `ssthresh = \max(FlightSize/2, 2 \times SMSS)`，把 `cwnd` 降低到不超过一个 `SMSS`，然后重新慢启动；

6. 实现三次重复 ACK 丢包响应：执行第二阶段的基本快速重传，按上述原则设置 `ssthresh` 并降低 `cwnd`；若未选做完整快速恢复，可以在重传数据得到新 ACK 后令 `cwnd=ssthresh` 并进入拥塞避免；
7. 确认 SYN 或 SYN-ACK 不增加 `cwnd`；
8. 输出能够反映 ACK、重复 ACK、RTO、重传、状态切换和窗口更新的 trace。

测试与判读

至少设计以下测试：

1. 无丢包、接收窗口足够大：观察 `cwnd` 先按慢启动增长，再进入拥塞避免；
2. 人为制造 RTO：检查 `ssthresh` 依据 `FlightSize` 降低，`cwnd` 回到不超过一个 `SMSS`；
3. 制造三次重复 ACK：检查基本快速重传和窗口降低；
4. 设置较小 `rwnd`：检查即使 `cwnd` 较大，在途数据仍受 `rwnd` 限制；
5. 对照 trace 与代码逐次核算关键窗口更新，确认图表不是仅根据期望行为人工绘制。

6.3.2 子任务二：联调、功能测试与性能评价

测试设计

至少从以下因素中选择两类开展对照实验：

- 丢包率；
- 网络时延；
- 带宽；
- 发送/接收缓冲区大小；
- 报文数据长度；
- 是否启用某项协议机制。

主要指标包括吞吐率；也可以考察完成时间、重传次数、有效数据比例、`rwnd`、`cwnd` 和 `ssthresh` 变化等。基础 Reno 已经是必做功能，因此性能测试应在启用基础 Reno 的最终代码上进行。

实验要求

1. 每组对照实验只改变计划研究的变量；
2. 记录代码版本、环境和全部参数；
3. 说明每种配置的重复次数和统计方法；
4. 保存原始数据、日志、trace 和绘图脚本；
5. 图表应包含完整坐标名称、单位、图例和必要说明；
6. 不仅描述变化趋势，还应从协议机制和代码实现解释原因；
7. 分析异常结果、局限性和可能的改进方法；
8. AI 可以辅助生成绘图代码和提出解释假设，但不得生成或补造实验数据。

6.3.3 子任务三：最终报告、代码复现与个人答辩

1. 整理并提交最终可编译源代码、构建与运行说明和最终 Git 版本；
2. 提交功能与性能测试的原始数据、日志、trace 和绘图脚本；
3. 按第 7 节要求完成最终课程报告和 AI 使用说明；
4. 复现主要功能和性能实验，确认报告中的图表可由提交材料生成；
5. 准备个人展示材料，按第 8.5 节要求参加答辩；
6. 根据要求完成代码定位、局部修改或补充测试。

6.3.4 第三阶段提交材料

第 3 周周末统一提交：基础 Reno 源代码和测试证据、最终代码和 Git 版本、构建与运行说明、功能和性能测试材料、原始数据与绘图脚本、最终课程报告、AI 使用说明和个人展示材料。个人答辩的具体时间和形式以课程通知为准。

6.4 挑战任务：高级拥塞控制与丢包恢复

挑战任务独立于三个阶段的基本分值。学生完成基础 Reno 后，可以选择以下一项或多项：

1. **完整 Reno 快速恢复**：依据 RFC 5681 实现第三个重复 ACK 后的窗口膨胀、后续重复 ACK 期间的新数据发送，以及恢复 ACK 后的窗口收缩；
2. **NewReno**：依据 RFC 6582 处理快速恢复中的部分确认，验证单个窗口内多报文丢失的恢复过程；
3. **SACK 及 SACK 丢包恢复**：依据 RFC 2018 实现 SACK 协商和处理，维护记分板，并参考 RFC 6675 进行丢包恢复；
4. **RACK-TLP**：依据 RFC 8985 使用逐报文发送时间、SACK 和探测报文进行基于时间的丢包检测；该方向以前置实现 SACK 和必要时间信息为条件；
5. **CUBIC**：依据 RFC 9438 实现 CUBIC 窗口增长和拥塞响应，并与基础 Reno 比较。

每项挑战任务均应提交：

- RFC 依据和与基础 Reno 的差异说明；
- 数据结构、状态和关键算法设计；
- 独立代码分支或可明确识别的 Git 提交；
- 正常、丢包、重排或高带宽时延积等与机制相匹配的测试；
- 原始 trace、绘图脚本、结果图和对比分析。

挑战实现不得破坏已经通过的连接管理、可靠传输、流量控制和基础 Reno 功能。

多个挑战方向存在依赖关系，例如 RACK-TLP 依赖 SACK 和逐报文时间信息；选择前应先核对前置条件和三周之外的工作量。

7. 报告与提交要求

7.1 进度报告

第一阶段、第二阶段和第三阶段分别在三周的周末提交累积更新的进度报告。每次报告覆盖该阶段全部子任务，至少包含：

1. 本阶段任务及完成情况；
2. 对应设计内容；
3. 主要代码实现说明，禁止大段粘贴源代码；
4. 测试环境、方法和结果；
5. 遇到的问题、原因分析和解决过程；
6. 本阶段代表性 AI 使用及验证记录；
7. 对应 Git 提交标识；
8. 尚未完成内容和下一阶段安排；第三阶段报告改为总结和后续改进建议。

7.2 最终课程报告

最终报告按照课程模板撰写，建议包含：

1. 报告摘要；
2. 任务分析与标准依据；
3. 已有编程框架的架构分析；
4. 协议总体设计；
5. 连接管理设计与实现；
6. 可靠数据传输设计与实现；
7. 流量控制设计与实现；
8. 基础 Reno 拥塞控制设计与实现；
9. 功能测试、性能测试与结果分析；
10. 典型问题及调试过程；
11. 挑战任务设计、实现与对比分析（如选做）；
12. AI 辅助过程及人工验证；
13. 总结；
14. 参考文献；
15. 附录：AI 使用说明、主要 Git 提交和测试环境。

7.3 写作和格式要求

1. 使用指定模板，章节完整、结构清晰；
2. 采用专业、客观、简洁的技术语言，避免口语化表达；
3. 设计、实现和测试部分应一一对应；
4. 使用状态机图、流程图、架构图或伪代码解释关键机制，禁止大段粘贴源码；
5. 图置于相关正文附近，图号和图名置于图下方；
6. 表格应有表号和表名，置于表格上方；
7. 所有图表必须清晰，并能够追溯到本人代码和实验数据；

8. 正文引用的标准、书籍或其他材料应在参考文献中给出完整信息；
9. AI 生成或修改的内容必须由学生核验，不得包含虚构的标准条款、引用或数据；基础 Reno 不得误写成完整 Reno，挑战任务不得把未实现机制写入成果；
10. 报告文件、代码和附件的命名方式以课程平台要求为准。

7.4 源代码提交

源代码单独提交，不粘贴在报告正文中。代码包至少包含：

- 完整可编译源代码；
- 构建和运行说明；
- 必要的测试脚本；
- 性能实验的数据处理或绘图脚本；
- 课程要求的 Git 记录或版本说明；
- 不包含编译产物、无关大文件、账号、密钥或个人敏感信息。

8. 考核与评分

课程设计占课程总成绩的 100%。考核按照课程大纲中课程目标 1、2、3 分别占 30%、50%、20%的比例组织。

8.1 成绩构成

课程目标	考核内容	比例
课程目标 1	代码架构分析	10%
课程目标 1	任务与标准分析	5%
课程目标 1	协议总体及分模块设计	15%
课程目标 2	连接管理功能实现	10%
课程目标 2	可靠数据传输功能实现	15%
课程目标 2	流量控制功能实现	5%
课程目标 2	基础 Reno 拥塞控制功能实现	10%
课程目标 2	性能测试与结果分析	5%
课程目标 2	代码质量、版本管理与可复现性	5%
课程目标 3	最终课程报告	10%
课程目标 3	个人展示、答辩及代码讲解/修改	10%
	合计	100%

高级拥塞控制与丢包恢复挑战任务可以获得**不超过 10 分**的加分，课程最终成绩不超过 100 分。完整 Reno 快速恢复、NewReno、SACK/RACK 和 CUBIC 的具体加分应结合难度、完整性和实验质量确定，以当期课程通知为准。

8.2 三阶段过程考核

三个正式阶段在总成绩中的对应关系如下：

正式阶段	主要考核内容	对应成绩比例
第一阶段	代码架构分析 10%+任务与标准分析 5%+协议设计 15%	30%
第二阶段	连接管理 10%+可靠数据传输 15%+流量控制 5%	30%

第三阶段	基础 Reno10%+性能测试与结果分析 5%+代码质量、版本管理与可复现性 5%+最终报告 10%+个人答辩 10%	40%
	合计	100%

第一阶段主要依据架构分析、设计文档、基线运行和过程材料评价；第二阶段主要依据线上自动测试，同时结合本地测试、进度报告、提交时间和过程材料；第三阶段依据基础 Reno 自动测试与 trace、最终代码、可复现实验、性能分析、课程报告和个人答辩综合评价。

8.3 AI 辅助能力的评价

AI 能力不单独按“使用量”计分，而是嵌入各项评价：

- 架构分析：AI 结论是否有代码或运行证据；
- 协议设计：是否能够发现并修正 AI 遗漏的边界条件；
- 编程实现：AI 生成代码是否经过审查和测试；
- 测试分析：是否通过实验验证 AI 提出的故障假设；
- 基础拥塞控制：是否能够用 RFC、代码和 trace 解释每次关键窗口更新；
- 课程报告：是否如实说明 AI 参与并核实全部内容；
- 答辩：是否能够解释、修改和测试 AI 辅助生成的内容。

无法解释本人提交代码、AI 记录明显失实或关键结果无法复现，将影响相应考核项目成绩。

8.4 课程报告评分要点

内容	比例
按模板完整撰写各部分，设计、实现和测试相互对应	50%
技术内容正确、重点突出、逻辑清晰	20%
架构图、FSM、流程图、表格和实验图表规范且可追溯	15%
AI 使用与人工验证说明真实、简洁、充分	10%
语言和格式规范，参考文献完整	5%
合计	100%

8.5 个人展示与答辩

每名学生独立制作幻灯片并进行讲解。课程可以采用个人视频展示、现场答辩或二者结合的形式。

展示内容包括：

- 框架架构分析；
- 协议设计；
- 主要实现；
- 基础 Reno 的慢启动、拥塞避免和丢包后窗口降低；
- 功能和性能测试结果；
- 典型问题及调试过程；
- AI 辅助中发现并修正的代表性问题；
- 总结与局限性。

如采用视频：

- 分辨率不低于 720p；
- MP4 格式；
- 图像稳定、文字清晰、声音清楚；
- 建议时长 5 分钟以内；
- 必须由学生本人独立讲解。

答辩评分标准：

内容	比例
正确讲解框架、设计、实现和测试结果	50%
讲解思路清晰，语言流畅，表达准确	20%
幻灯片制作美观大方，图文并茂，重点突出，言简意赅	20%
视频录制满足要求	10%
合计	100%

无故不参加个人答辩，答辩部分记 0 分。

9. 独立完成与学术诚信

9.1 允许的交流

学生之间可以交流：

- TCP 和可靠传输的基本原理；
- RFC 阅读方法；
- Linux、Git、抓包和绘图工具的通用使用方法；
- 不涉及具体实现的调试思路；
- AI 工具的一般使用经验和数据安全注意事项。

9.2 禁止行为

禁止：

1. 交换、复制或共同编写源代码；
2. 交换完整设计文档、报告、AI 生成结果或实验数据；
3. 将他人代码或报告交给 AI 改写后提交；
4. 伪造 Git 记录、测试结果、日志、抓包、实验数据或 AI 记录；
5. 修改测试程序、固定返回值或识别测试用例以绕过验收；
6. 委托他人或 AI 代理完成课程平台提交和个人答辩；
7. 隐瞒对核心代码和文档有实质影响的 AI 使用；
8. 公开或传播课程框架、隐藏测试、教师参考实现和学生源码。

9.3 AI 工具和源码安全

课程源码在课程结束后三年内不得公开；如学校另有规定，以学校规定为准。优先使用学校批准的 AI 工具或自研课程助教智能体。未经许可，不得向公共 AI 服务上传：

- 完整课程框架或本人完整实现；
- 教师参考实现和隐藏测试；
- 课程平台账号、密钥；
- 其他学生的代码和报告；
- 含有个人信息或其他敏感数据的文件。

如确需使用外部 AI 服务，应只提供解决当前问题必需且已经脱敏的最小代码片段。

9.4 诚信认定

源代码和报告可以使用查重或相似性检测工具检查。

抄袭、代做、伪造和绕过测试等行为按照学校相关规定处理。学生不能以“代码或文字由 AI 生成”为理由免除责任。

10. 常见问题处理原则

1. **环境无法运行**：先核对框架版本、运行环境和命令，再检查网络、权限和依赖；不要在环境问题未解决时大规模修改代码。
2. **自动测试与本地结果不一致**：记录两侧环境、代码版本、参数和日志，构造最小复现实例。
3. **AI 给出多种互相冲突的方案**：回到 RFC、说明书和框架接口，以可验证的需求和测试结果作出选择。
4. **AI 生成代码无法解释**：不要提交；应缩小问题、重新分析并自行重写到能够理解和测试的程度。
5. **阶段任务未完成**：按时提交当前代码、失败测试、原因分析和下一步计划，不得伪造通过结果。
6. **性能结果异常**：重复实验，检查变量控制、计时方法、网络配置、系统负载和代码版本。
7. **拥塞窗口行为异常**：先区分 `rwnd`、`cwnd` 和 `FlightSize`，逐条核对 ACK、重复 ACK 和 RTO 事件，不要只根据最终吞吐率判断实现正确性。
8. **挑战任务工作量失控**：先完成并冻结基础 Reno 版本，再从独立分支选择一个挑战方向；RACK-TLP 等存在明确前置依赖，不宜在基础功能未通过时开始。

11. 参考资料

1. J. Kurose and K. Ross, *Computer Networking: A Top-Down Approach*, 8th Edition, Pearson, 2021.
2. RFC 9293, *Transmission Control Protocol (TCP)*, <https://www.rfc-editor.org/rfc/rfc9293.html>.
3. RFC 6298, *Computing TCP's Retransmission Timer*, <https://www.rfc-editor.org/rfc/rfc6298.html>.
4. RFC 5681, *TCP Congestion Control*, <https://www.rfc-editor.org/rfc/rfc5681.html>.
5. RFC 2018, *TCP Selective Acknowledgment Options*, <https://www.rfc-editor.org/rfc/rfc2018.html>.
6. RFC 6582, *The NewReno Modification to TCP's Fast Recovery Algorithm*, <https://www.rfc-editor.org/rfc/rfc6582.html>.
7. RFC 6675, *A Conservative Loss Recovery Algorithm Based on Selective Acknowledgment (SACK) for TCP*, <https://www.rfc-editor.org/rfc/rfc6675.html>.
8. RFC 7323, *TCP Extensions for High Performance*, <https://www.rfc-editor.org/rfc/rfc7323.html>.
9. RFC 8985, *The RACK-TLP Loss Detection Algorithm for TCP*, <https://www.rfc-editor.org/rfc/rfc8985.html>.
10. RFC 9438, *CUBIC for Fast and Long-Distance Networks*, <https://www.rfc-editor.org/rfc/rfc9438.html>.
11. RFC 793, *Transmission Control Protocol*, <https://www.rfc-editor.org/rfc/rfc793.html>.

12. W. Richard Stevens, *TCP/IP Illustrated, Volume 1: The Protocols*.
13. Gary R. Wright and W. Richard Stevens, *TCP/IP Illustrated, Volume 2: The Implementation*.
14. TCConfig, <https://github.com/thombashi/tcconfig>.
15. TCConfig Documentation, <https://tcconfig.readthedocs.io/>.